

Phase 6 - Deliverables & Reflection

Required Deliverables

A. Configuration Artifacts

Item	Evidence Command	Status
Parameter Store: endpoint	<code>aws ssm get-parameter --name /lab/db/endpoint</code>	[]
Parameter Store: port	<code>aws ssm get-parameter --name /lab/db/port</code>	[]
Parameter Store: name	<code>aws ssm get-parameter --name /lab/db/name</code>	[]
Secrets Manager secret	<code>aws secretsmanager get-secret-value --secret-id lab/rds/mysql</code>	[]

B. Observability Proof

Item	Evidence Command	Status
Log group exists	<code>aws logs describe-log-groups --log-group-name-prefix /aws/ec2/lab-rds-app</code>	[]
ERROR logs captured	<code>aws logs filter-log-events --log-group-name /aws/ec2/lab-rds-app --filter-pattern "ERROR"</code>	[]
Alarm configured	<code>aws cloudwatch describe-alarms --alarm-name-prefix lab-db-connection</code>	[]

C. Incident Response Proof

Item	Evidence	Status
Simulated failure	Screenshot/output of failed <code>curl</code> request	[]
Alarm triggered	<code>aws cloudwatch describe-alarm-history --alarm-name lab-db-connection-failure-alarm</code>	[]
Successful recovery	Screenshot/output of working <code>curl</code> after fix	[]

Item	Evidence	Status
Alarm cleared	Alarm state shows OK	[]

Reflection Questions (Must Answer All)

A) Why might Parameter Store still exist alongside Secrets Manager?

 Click to reveal model answer

Key Points:

- **Cost:** Parameter Store standard parameters are **free**. Secrets Manager charges ~\$0.40/secret/month.
- **Rotation:** Secrets Manager has **built-in automatic rotation** with Lambda integration. Parameter Store doesn't.
- **History:** Parameter Store existed first (2016); Secrets Manager came later (2018) specifically for credentials.
- **Use case fit:** Parameter Store for config (endpoints, feature flags), Secrets Manager for credentials (passwords, API keys).
- **Many orgs use both intentionally** — Parameter Store for non-sensitive config, Secrets Manager for secrets.

B) What breaks first during secret rotation?

 Click to reveal model answer

The application layer breaks first. Here's why:

1. Secrets Manager rotates the password (updates the secret)
2. Lambda updates the password on the database side
3. **But the application still has the OLD password cached in memory**
4. App tries to connect → "Access denied" → Outage

This is exactly what Scenario C teaches. Apps must re-fetch credentials on connection failure (or use short TTL caching).

C) Why should alarms be based on symptoms instead of causes?

▶ Click to reveal model answer

- **Symptom-based alarm:** "DB connections are failing" → Catches ANY cause
- **Cause-based alarm:** "Security Group was modified" → Only catches ONE specific cause

Why symptoms win:

1. You can't predict every failure mode
2. A single symptom alarm catches multiple root causes
3. Detection comes first, diagnosis comes second
4. Users don't care WHY it's broken — they care that it IS broken

Your lab example: Your alarm fired on **ERROR** logs (a symptom). It would have fired whether the cause was SG blocking, RDS stopped, or bad credentials.

D) How does this lab reduce mean time to recovery (MTTR)?

▶ Click to reveal model answer

Component	How it reduces MTTR
CloudWatch Alarm	Faster detection — you know immediately, not after user complaints
CloudWatch Logs	Faster diagnosis — error messages tell you what's wrong
Parameter Store	Faster recovery — known-good config is pre-stored, no guessing
Secrets Manager	Faster recovery — correct credentials are retrievable instantly
This runbook	Faster response — documented steps mean no improvisation

MTTR = Detection + Diagnosis + Recovery. This lab optimized all three.

E) What would you automate next?

▶ Click to reveal model answer

Strong answers:

- **Terraform:** Codify all resources for repeatable deployment

- **Auto-remediation Lambda:** Triggered by CloudWatch Alarm to automatically restore SG rules
 - **Automated secret rotation:** Secrets Manager can rotate credentials on a schedule
 - **CI/CD pipeline:** Auto-deploy app changes with rollback on failure
 - **Systems Manager Automation:** Runbooks that execute recovery steps automatically
-

Evidence Collection Script

Run this to generate all evidence outputs:

```
#!/bin/bash
echo "=== PHASE 1: Configuration Artifacts ==="
echo "--- Parameter Store ---"
aws ssm get-parameters --names /lab/db/endpoint /lab/db/port
/lab/db/name --with-decryption

echo "--- Secrets Manager ---"
aws secretsmanager get-secret-value --secret-id lab/rds/mysql

echo ""
echo "=== PHASE 2: Observability ==="
echo "--- Log Group ---"
aws logs describe-log-groups --log-group-name-prefix /aws/ec
2/lab-rds-app

echo "--- ERROR Logs ---"
aws logs filter-log-events --log-group-name /aws/ec2/lab-rds-
app --filter-pattern "ERROR" --limit 5

echo ""
echo "=== PHASE 3: Alarm ==="
aws cloudwatch describe-alarms --alarm-name-prefix lab-db-con
```

```
nection
```

```
echo ""
```

```
echo "=== Alarm History ==="
```

```
aws cloudwatch describe-alarm-history --alarm-name lab-db-connection-failure-alarm --history-item-type StateUpdate --max-records 5
```

What Completing Lab 1B Proves



If you complete this lab, you can confidently say:

"I can operate, monitor, and recover AWS workloads using proper secret management and observability."

That is mid-level engineer capability, not entry-level.

Skills Demonstrated

Skill Area	What You Proved
Secret Management	Know where secrets live, why multiple systems exist
Observability	Diagnose outages using logs and metrics
Incident Response	Respond without redeploying everything
MTTR Reduction	Restore service quickly using known-good configuration